NEXTGEN QA

NextGen QA

An AI-powered system that transforms Agile conversations into comprehensive, self-healing test suites



PROJECT R26-SE-039 | SPECIALISATION SOFTWARE ENGINEERING

PP2 panel comments and actions taken

PP2 panel comment	Action taken	Evidence / status
IT22303684: Independently validate the generated test cases.	Deterministic checks plus a separate LLM judge, with reviewable findings and proposed fixes.	Tested on real stored suites; validator matched real CI results 86 % of the time.

OTHER THREE MEMBERS: NO SPECIFIC PP2 COMMENTS

Design excellence over existing solutions

THE PROBLEM

- Meeting requirements, test design and execution are disconnected.
- Failed CI/CD tests require manual investigation and repair.
- Test generation can produce invalid selectors and miss important scenarios.
- Test quality, coverage, and traceability become outdated as software changes.

NEXTGENQA INTRODUCE

- Connects meeting requirements, user stories, test generation, and execution in one workflow.
- Uses ML to identify failure causes and supports developer-approved, controlled repairs.
- Generates tests using actual DOM elements and explores the application to discover additional scenarios.
- Continuously updates test quality, coverage gaps, and the RTM using actual execution evidence.

Intelligent User Story Generator & Conflict detector



IT22251114 | Jinad Induvidwa A.G

INPUT

Live meeting audio / uploaded **text transcripts**.

PROCESS

Identifies requirements using **ModernBERT classification**, detect requirement conflict via **semantic vector analysis**, generates user stories using **RAG**, and passes them through a 5-layer validation engine (**syntax rules**, **evidence grounding**, **hallucination detection**, **INVEST scoring**).

OUTPUT

Formatted Agile user stories with Gherkin **acceptance criteria**, detailed **5-layer quality reports**, and **direct synchronization with Jira**

Intelligent Test Case Generation



IT22303684 | Abeygunasekara D.T

INPUT

Approved **user stories** and acceptance criteria imported from the story generator, the target **web site URL**, and the chosen **test framework** (Playwright, Selenium or Cypress)

PROCESS

Gherkin generation from each story → DOM crawler collects the real page selectors → AI agent explores the site and finds user flows → multi-framework code generation using those real selectors → independent validation by rule checks and a separate AI judge → ML risk score for each flow

OUTPUT

Versioned **runnable test suites** for the selected framework, **CI/CD execution** results with screenshots and logs, a downloadable PDF report, a validation report with a score and verdict, and a risk score that tells QA which flows to test first

component eke nama da gnn



IT22371768 | Ranasinghe S.I

INPUT **Failed CI jobs** and diagnostic **logs**

PROCESS Nine-class **TF-IDF + Logistic Regression**, deterministic **repair policy**, **two-stage developer approval**

OUTPUT **Failure** diagnosis, **non-write action** or reviewed **draft-PR proposal**

ML-Based Quality Prediction & Requirements Traceability Matrix (RTM)



IT22182678 | Rashani K.G.M.

INPUT

Requirements, **generated tests** and e**xecution/coverage evidence**

PROCESS

Builds a **live RTM** on every request, **predicts test-case quality** before execution using a **Random Forest** model (10 Gherkin features), **detects acceptance-criterion-level coverage gaps**, and ranks them with an explainable 5-factor risk score. It also **generates tests for uncovered criteria and improves weak tests**.

OUTPUT

Traceability gaps, **Quality Scores** and **A live traceability matrix**, High / Medium / Low quality scores, **risk-ranked coverage gaps** with recommended actions

Results, evaluation evidence, limitations and future improvement

EVALUATION EVIDANCE

- **User story generation:** Compared three LLMs using the same meeting transcript and seven ground-truth user stories.
- **Test Case generation:** Compared DOM-grounded and abstract test generation using executability and selector accuracy..
- **Root-cause classification:** Evaluated agentic exploration against random exploration and breadth-first search.
- **RTM generation:** Measured repair safety, RTM correctness, update latency, and coverage-agent success.

RESULTS

- **User story generation:** Claude Sonnet LLM Model achieved 85.71% requirement coverage.
- **Test Case generation:** test generation achieved 91.2% executability and 93.5% selector accuracy.
- **Root-cause classification:** achieved a macro-F1 score of 0.89.
- **RTM generation:** status correctness reached 98.7%, with 143 ms update latency.

USER FEEDBACK

LIMITATION & FUTURE IMPROVEMENT

- Results may vary across applications, domains, and technology stacks.
- Real-world practitioner studies and long-term production evaluations are still needed.
- Total pipeline latency, operational costs, and scalability require further evaluation.